Assignment 14:

```
/**
 *
================================================================================
 * KBTCOE Connect — Backend REST API Server (server.js)
 * Assignment 14: Node.js and Express Server
 *
================================================================================
 */

'use strict';

const path = require('path');
const express = require('express');

// Middleware imports
const notFound = require('./middleware/notFound');
const errorHandler = require('./middleware/errorHandler');

// Route imports
const opportunitiesRouter = require('./routes/opportunities');

// Initialize Express application
const app = express();

// Configurable listening port via environment variable or default 3000
const PORT = process.env.PORT || 3000;

// Application metadata
const APP_METADATA = {
  name: 'KBTCOE Connect REST API',
  version: '1.0.0',
  description: 'Assignment 14: Node.js & Express REST Backend',
  department: 'Department of Information Technology, KBTCOE',
  startTime: new Date().toISOString()
};

//
================================================================================
// 1. Built-in and Custom Middleware
//
================================================================================

/**
 * Custom Request Logger Middleware
 * Logs HTTP method, URL, status code, and execution duration.
 */
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
```

```
    const duration = Date.now() - start;
    const logTime = new Date().toISOString().replace('T', ' ').slice(0, 19);
    console.log(
      `[${logTime}] ${req.method} ${req.originalUrl} -> ${res.statusCode}
(${duration}ms)`
    );
  });
  next();
});

// Parse application/json request bodies (with malformed JSON caught by
errorHandler)
app.use(express.json());

// Parse application/x-www-form-urlencoded payloads
app.use(express.urlencoded({ extended: true }));

// Serve static assets from public/ directory
app.use(express.static(path.join(__dirname, 'public')));

//
================================================================================
// 2. Application Routes
//
================================================================================

/**
 * GET /api/health
 * Returns service health status, uptime, and metadata.
 */
app.get('/api/health', (req, res) => {
  const uptimeSeconds = Math.floor(process.uptime());
  res.status(200).json({
    success: true,
    message: 'KBTCOE Connect REST API is healthy and operational',
    data: {
      status: 'UP',
      service: APP_METADATA.name,
      version: APP_METADATA.version,
      environment: process.env.NODE_ENV || 'development',
      port: Number(PORT),
      uptimeSeconds,
      uptimeFormatted: `${Math.floor(uptimeSeconds / 60)}m ${uptimeSeconds %
60}s`,
      serverTime: new Date().toISOString()
    }
  });
});

// Mount Opportunity resource endpoints
app.use('/api/opportunities', opportunitiesRouter);

//
================================================================================
```

```
// 3. Error Handling Middleware (Order of definition is critical)
//
================================================================================

// Handle 404 for unmapped endpoints
app.use(notFound);

// Centralized error handler (catches JSON parse errors & internal exceptions)
app.use(errorHandler);

//
================================================================================
// 4. Server Initialization
//
================================================================================

let server = null;

// Only start listening if executed directly (enables test imports without port
binding)
if (require.main === module) {
  server = app.listen(PORT, () => {

console.log('================================================================');
    console.log('🚀 KBTCOE Connect Backend Express Server is running!');
    console.log(`📡 Base URL:          http://localhost:${PORT}`);
    console.log(`🏥 Health Check:      http://localhost:${PORT}/api/health`);
    console.log(`📦 Opportunities API:
http://localhost:${PORT}/api/opportunities`);
    console.log(`🎨 Test UI Web App:   http://localhost:${PORT}/`);
    console.log(`🕒 Started at:        ${new Date().toISOString()}`);

console.log('================================================================');
  });

  // Graceful shutdown handling
  const shutdown = (signal) => {
    console.log(`\n[SHUTDOWN] Received ${signal}. Closing HTTP server...`);
    server.close(() => {
      console.log('[SHUTDOWN] HTTP server closed gracefully. Exiting process.');
      process.exit(0);
    });
  };

  process.on('SIGINT', () => shutdown('SIGINT'));
  process.on('SIGTERM', () => shutdown('SIGTERM'));
}

module.exports = { app, server, PORT };
```